

The Ethics of Generative AI in Commercial Software Development

Roman Marquez-Twisdale
r.marqueztwisdale@digipen.edu

ACM Reference Format:

Roman Marquez-Twisdale. 2026. The Ethics of Generative AI in Commercial Software Development. In . ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Over the past few years, generative AI has started to become a normal part of software development. Models capable of generating code from simple prompts are now used by students, developers, and large companies working on commercial software. In some ways this seems like a huge benefit. Developers can work more efficiently in a lot of cases. Due to this, it's not surprising that many software companies would start to partially rely on AI-generated code in the development of their real-world applications. But just because something is useful doesn't make it ethical, especially if that code is being shipped in commercial products.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1.0.1 Irresponsible Development. One of the biggest issues is how it's being used. In many cases, developers might not fully understand the code that the AI produces, yet still include it in finished products. This makes it more likely that something will break or introduces a security issue. There's also the problem of transparency. If developers don't acknowledge when or how AI was used, how do we know who wrote a piece of software? How do we know who owns a piece of software? These problems are only amplified in commercial settings, where software affects real users.

1.0.2 Copyright Issues. There are larger concerns about where AI-generated code comes from. Since these systems are trained on large amounts of existing code, much of which was scraped from the internet, including open-source projects, they might reproduce patterns or structures without proper acknowledgement to the original authors.

1.0.3 Core Argument. This paper argues that using machine learning models to generate code for commercial applications is only ethical under certain conditions. Specifically, developers need to understand the code they use and be transparent about how AI contributed to the final product.

2 Background

To understand the ethical issues around AI-generated code in commercial software, let's clarify what is being discussed and who is affected. Software engineers range from individual developers working on small projects to large teams building complex systems. Generative AI refers to Large Language Models that produce code from natural language

prompts, often trained on massive datasets containing publicly available code and documentation. Commercial applications includes software sold on the open market, and internal systems which play any role in the development of the software being sold. The ethical concern arises because LLMs are not currently held responsible for the code they generate and neither is their creators. The generated code is not fully written by the developer and not clearly belonging to any single original source.

2.0.1 Highlights. Research has shown that generative AI systems can reproduce patterns from training data. "According to the award-winning website Copyleaks, nearly 60% of the responses provided by Defendants' GPT-3.5 product in a study conducted by Copyleaks contained some form of plagiarized content, and over 45% contained text that was identical to pre-existing content." [7]. These models are also known to produce inaccurate or misleading outputs, sometimes referred to as "hallucinations," which can make their use risky in real-world applications [3].

3 Stakeholders

Software engineers benefit from increased productivity but if not careful, can end up with software they don't fully understand. Companies gain efficiency but weaken their legal and ethical security, especially if AI-generated code includes copyrighted material without. "researchers warn that black-box AI tools may blend code from multiple sources, risking inadvertent violations" [9]. End users are also affected, even if indirectly. They depend on software to function as intended, bugs and vulnerabilities impact them directly. AI-generated content has already been seen in other domains, creating challenges for maintaining quality and trust in all communities. "It's unlikely that any human editor or technology can sustain an ability to differentiate human from machine writing." [10]. The open-source community is another important stakeholder. Many AI systems are trained on publicly available software, often without explicit consent from contributors. This raises concerns about

respect for intellectual labor. [7]. Finally, the organizations that develop AI systems also play a role. These organizations are at the forefront of ethical guidelines adoption. If they choose to emphasize transparency, and accountability, they could have a powerful impact on the emerging technology, but this may come at a cost. [4], [8].

4 Current State of Generative AI

Many developers now use LLMs for tasks like writing basic code, debugging, and documentation. Developers can generate functional code much faster than writing everything by hand. Research suggests these tools are particularly effective for documentation, testing, and basic implementation [6]. However, these benefits come with a cost. AI-generated code is not always correct and can contain logical errors or subtle bugs. Even when it appears correct, it may include poor design choices. Security is another concern. Research has found that AI-generated code can include unsafe practices, and many developers do not consistently review it before use [6]. There is also the issue of over-confidence. Developers might use AI outputs without fully checking them, even if they don't completely trust them. Over time, this can also affect skill development, as developers rely more on AI instead of problem-solving themselves [6]. Overall, generative AI is powerful but potentially dangerous. It works well for simple tasks but struggles with more complex problems, making human oversight essential for responsible development.

5 Technical Background

To understand why AI-generated code can be risky, it helps to have a basic idea of how these models work. Most modern generative AI tools are based on large language models, which are trained on massive datasets. During training, the model learns patterns in how words and code are structured, but

it is commonly believed that it doesn't actually "understand" what it is generating in a human sense [6].

5.0.1 High Level. These models work by breaking input text into smaller pieces called tokens and then predicting what token should come next based on a learned pattern. When a developer submits a prompt to a Large Language Model, the model processes it and generates code one token at a time using a Recurrent Neural Network (RNN), a type of neural network that feeds its own outputs back into itself in order to remember and use previous states to help predict future states. "While the 'meaning' of these tokens is implicit to some extent within these associations, the model is not explicitly utilising this meaning, but is rather generating syntactically appropriate strings of text based on statistical associations" [3].

One of the most common LLMs used today is ChatGPT. The acronym "GPT" stands for Generative Pre-Trained Transformer and represents one of the most common architectures used today. In 2017, the paper "Attention Is All You Need" was published describing this architecture.

5.0.2 Encoder. Within this model architecture, the inputs are encoded into value sequences and passed through a section of the network known as the Multi-Head Attention mechanism. They are then passed through a traditional feed forward neural network. After each of these steps there is a normalization process. Finally, the encoder produces an encodes sequence of values that can be passed through the decoder to produce the final result.

5.0.3 Decoder. The Decoder is fed all previous outputs along with the current encoded input. Both the previous outputs and the encoded input are passed through their own Multi-Head Attention mechanisms, and then combined to produce a final result for the next token in the sequence. It is the previous tokens which allow for this architecture to recognize patterns and adjust its behavior based on what came before. If we didn't pass in the previous token outputs during the computation of each

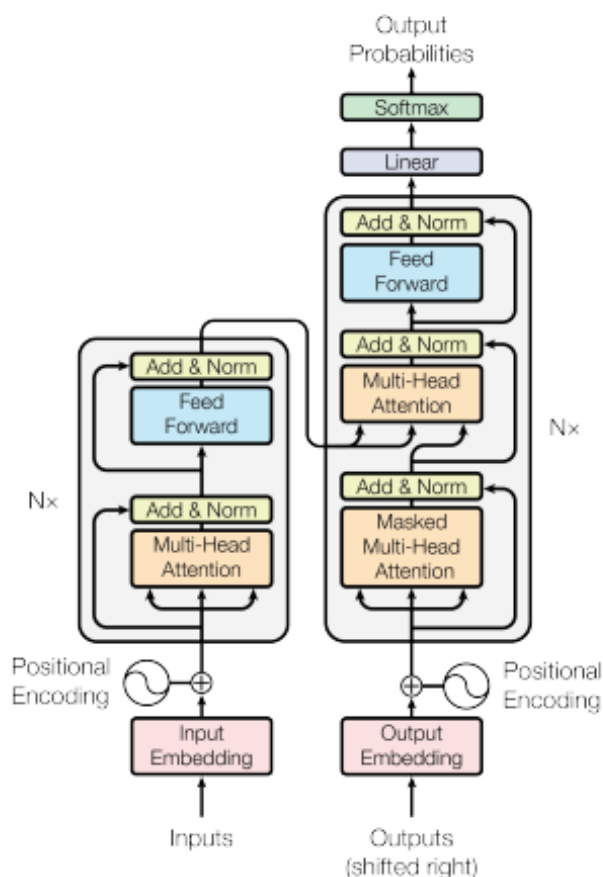


Figure 1: The Transformer - model architecture.

Figure 1: The Encoder-Decoder Transformer Architecture.

subsequent token, we would essentially be stripping the model of any long term memory. very likely, the model wouldn't be able to produce text longer than a few sentences.

5.0.4 Attention. The Multi-Head Attention

6 Ethical Analysis

Social Contract Theory provides a useful framework for evaluating this issue. It is based on the idea that society functions through shared expectations like honesty, fairness, and accountability. In

software development, there is an implicit agreement that developers understand and take responsibility for the code they produce. When developers rely on AI-generated code without fully understanding it, this weakens trust. If something goes wrong, responsibility becomes unclear, which conflicts with the expectation that developers stand behind their work. Transparency is another key part of this social contract. If developers do not acknowledge AI use, they may present the work as entirely their own, raising concerns about fairness and authorship. This is especially important given that AI systems are trained on other developers' contributions [7], [9]. There is also an implicit agreement with the open-source community. Contributors share their work expecting proper credit, but AI systems may produce similar code without attribution. This creates tension between convenience and fairness. However, Social Contract Theory does not imply that AI is always unethical. If developers review and understand the code they use, and if companies are transparent about AI involvement, they can still uphold these expectations. Ethical guidelines for AI emphasize transparency and accountability as key principles [4], [8].

7 Safeguards and Responsibilities

Given the current state of generative AI, the main issue is how it is used. Making AI-generated code ethical comes down to safeguards and responsibility. One of the most important safeguards is requiring developers to review and understand any AI-generated code before using it. Since AI can produce incorrect or misleading outputs, blindly trusting it can lead to bugs or security issues [3], [5]. Transparency is also essential. Developers and companies should acknowledge when AI tools are used, especially in commercial projects. Without this, it becomes harder to assign responsibility or evaluate originality [4], [8]. Companies also have a role in setting standards, such as requiring code reviews and training developers on risks. Since they benefit from AI, they are responsible for managing

its impact. Finally, the organizations that develop AI systems share responsibility for how models are trained and what data is used. Concerns about bias, copyright, and fairness suggest that these systems should be designed with ethical risks in mind [4], [8].

8 Conclusion

Generative AI is becoming a normal part of software development. It offers clear benefits in terms of productivity and accessibility, but that does not mean its use is always ethical. The main concerns are lack of understanding and lack of transparency. When developers use AI-generated code without fully understanding it, they take responsibility for something they did not completely create. When they fail to acknowledge AI use, it raises concerns about honesty and authorship. From an ethical perspective, developers still have a responsibility to ensure their code is correct and safe, regardless of how it was created. At the same time, misuse of AI could reduce trust in software systems and create unfair conditions for others [2], [8]. That said, generative AI can be used ethically if certain conditions are met. Developers need to understand and verify the code they use, companies need clear guidelines, and there needs to be transparency about AI's role. Without these safeguards, the risks can outweigh the benefits, especially in commercial applications. For me personally, this changes how I think about using AI tools in my own work. It's easy to rely on them for quick answers, but that doesn't mean I actually understand what I'm shipping. Going forward, I think the most important takeaway is that using AI responsibly means treating it as a tool, not a replacement for understanding. If developers can do that, then AI can be a helpful part of the development process. If not, it becomes a shortcut that creates more problems than it solves.

References